INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	*Computación gráfica, visión computacional y multimedia*				
TÍTULO DE LA PRÁCTICA:	*Motores de física y colisiones*				
NÚMERO DE PRÁCTICA:	*01*	**AÑO LECTIVO:**	*2026-A*	**NRO. SEMESTRE:**	*VII*
FECHA DE PRESENTACIÓN	*22/04/2026*	**HORA DE PRESENTACIÓN**	*12:20pm*		
INTEGRANTE (s): Perez Huamani Jeremy Joshua				**NOTA:**	
DOCENTE(s): Prof. René Nieto					

SOLUCIÓN Y RESULTADOS

I. EJERCICIOS PROPUESTOS

Carpeta con videos.

https://drive.google.com/drive/folders/145lXy7GcCjjkhQQFxo6mXWNVrOO4AO7F

Deberán crear un entorno que guarde relación con el proyecto que desean realizar.

Tendrán que diseñar un personaje controlable por el jugador con la capacidad de moverse y disparar los proyectiles.

La interacción del usuario se puede realizar mediante botones o al presionar teclas.

Deberán establecer como funcionarán las colisiones del proyectil por ejemplo en un juego de basquetbol cuando la pelota logre pasar por encima del aro le debe mostrar al usuario que logro obtener puntos por un correcto lanzamiento, pero si la colisión se realiza por debajo del aro no debería permitir ganar puntos o en el caso de un juego de disparo que solo afecte el disparo que llegue a un punto especifico del enemigo.

La aplicación debe permitir gestionar valores como la puntuación basada en base a las canastas obtenidas o la vida de los enemigos o del jugador.

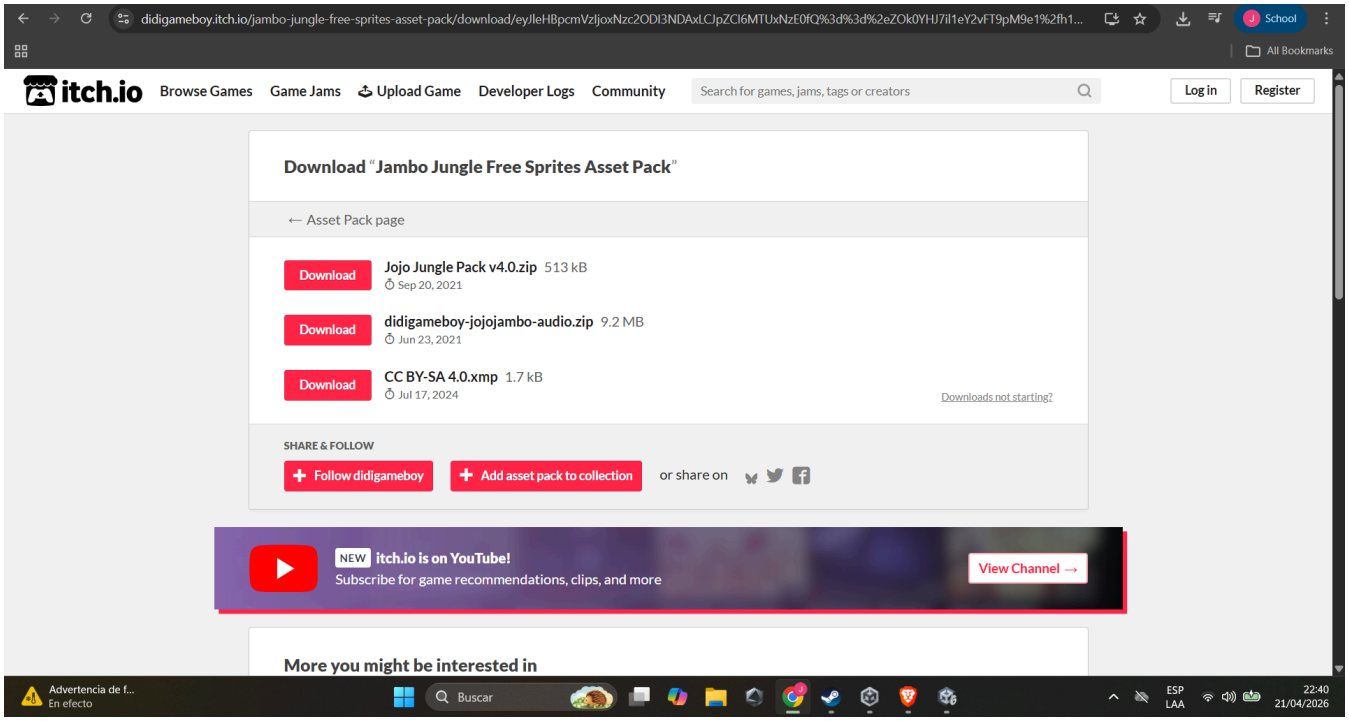
Se busca que investiguen que otros elementos pueden agregar a la aplicación que mejoren la experiencia del usuario por ejemplo efectos visuales y sonoros o implementar diferentes tipos de proyectiles, un sistema de niveles con objetivos más difíciles y enemigos con patrones de ataque más complejos, agrega power-ups que otorguen habilidades especiales al jugador o diseñar un sistema de inteligencia artificial para los enemigos que les permita esquivar los proyectiles del jugador.

Deben crear un informe donde describan las partes desarrolladas de la aplicación explicando el funcionamiento del código, se debe agregar imágenes.

Adicionalmente deben subir un video corto donde muestren el uso de la aplicación explicando lo realizado tanto en la creación del escenario, el manejo de las colisiones y las interacciones del usuario en el escenario.

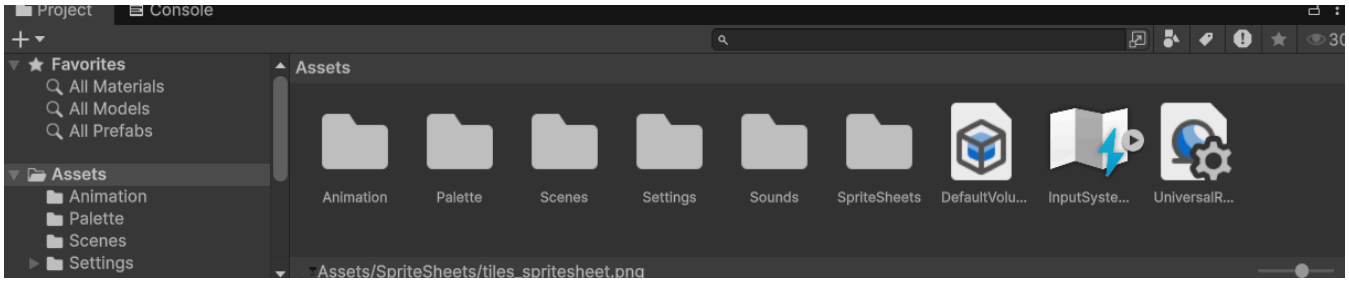
Opcionalmente pueden subir el código y/o ejecutable de su aplicación.

Para empezar, este juego lo realizaremos con unity, en un entorno 2D. Para los sprites utilizaremos los que se pueden encontrar en la siguiente página.



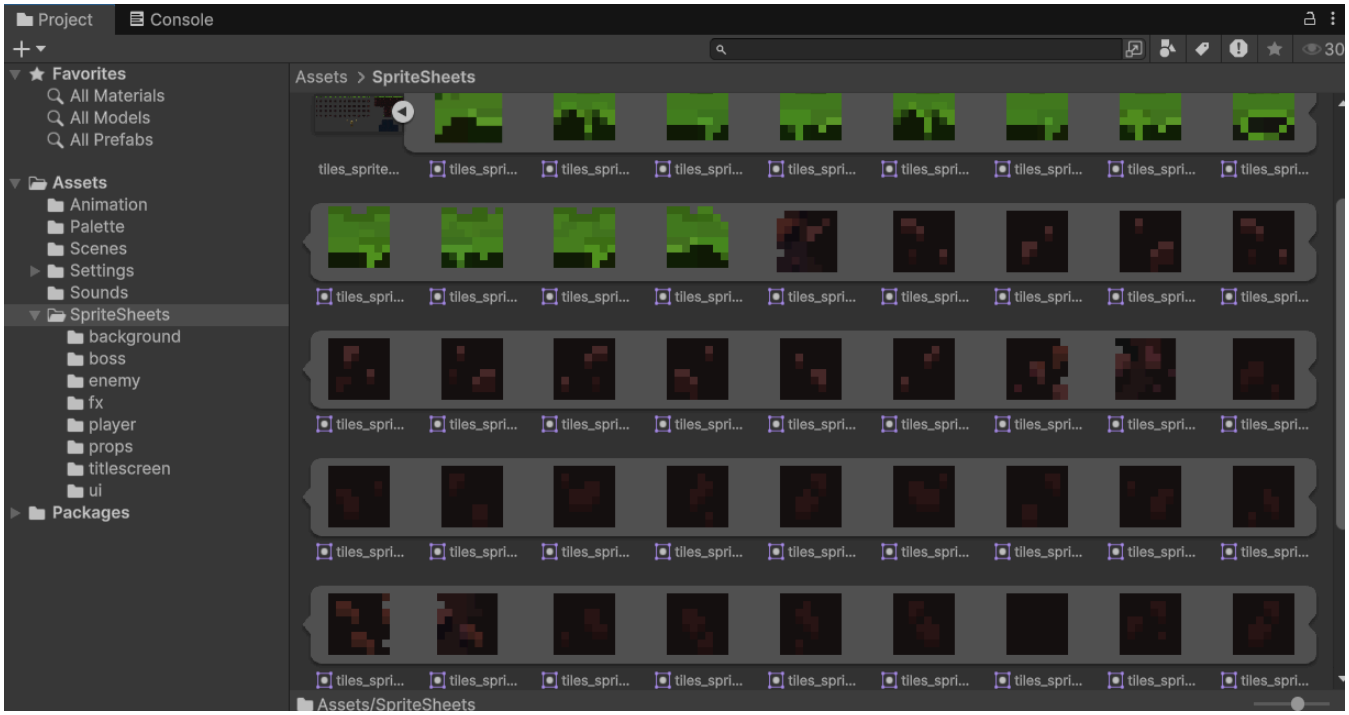
Esto con el fin de poder agilizar el proceso y no tener que hacer sprites desde 0.

Una vez hecho esto descomprimimos los archivos y los agregamos a nuestro entorno unity.

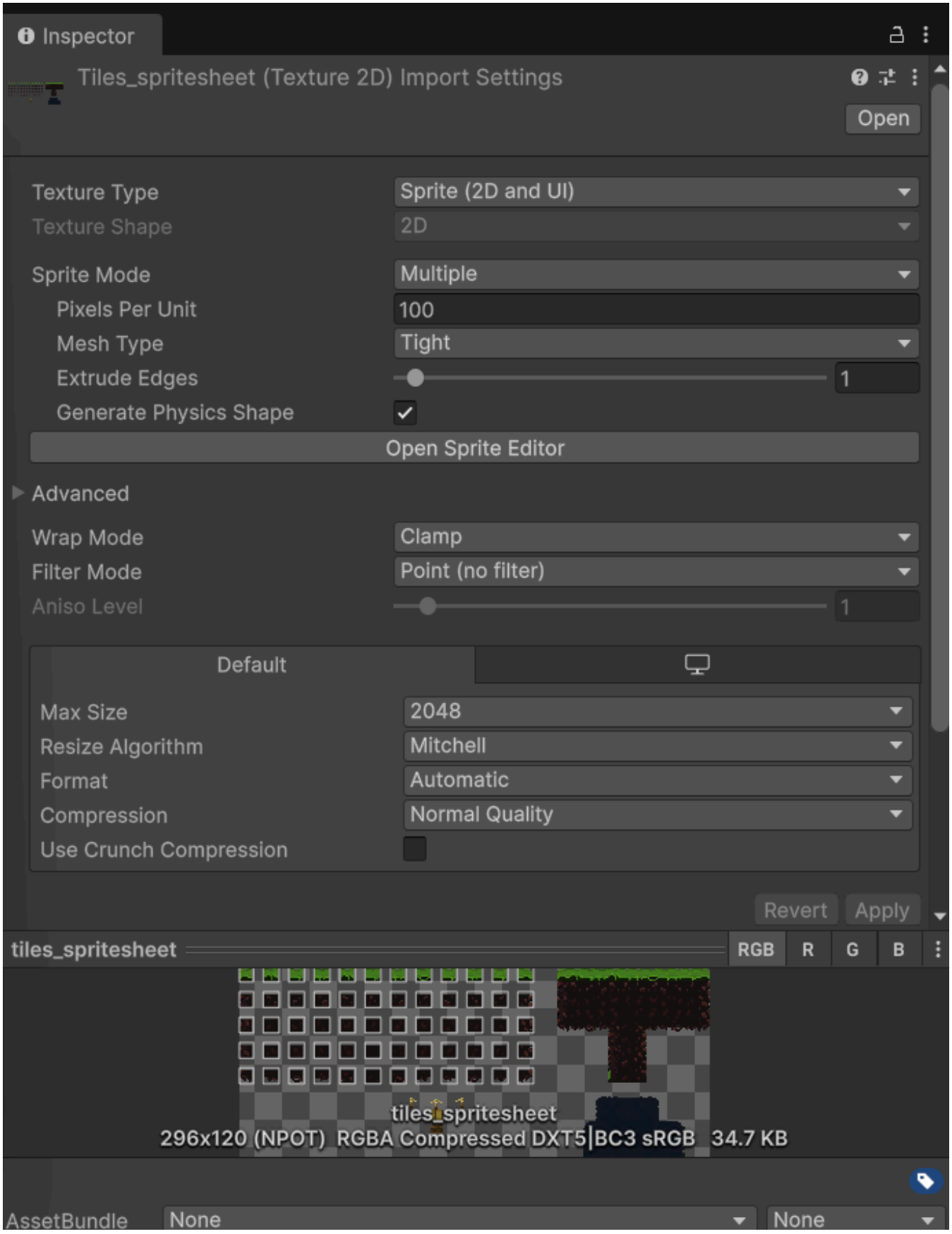


Estos se encuentra en la carpeta de Spritesheets y Palette.

Para crear el mundo inicial empezaremos con los sprites de para el tile.

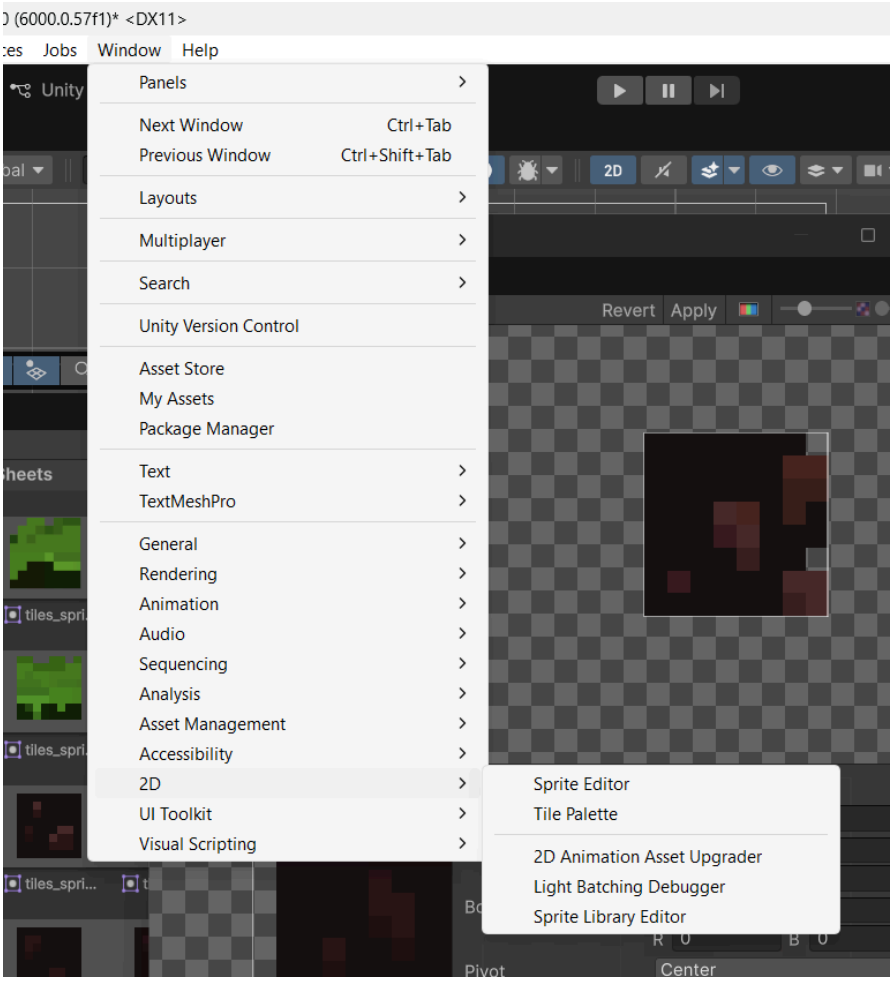


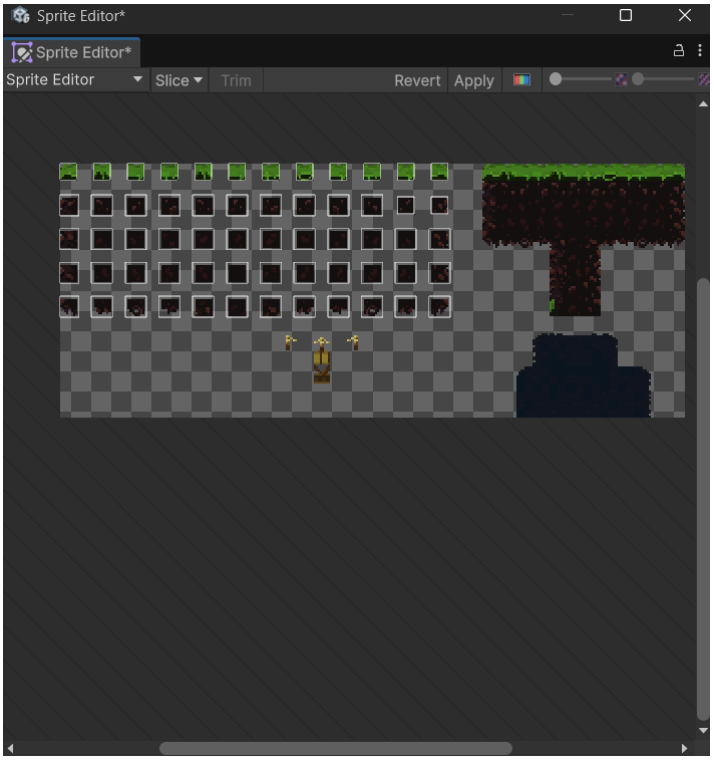
Seleccionamos el tiles sprite y nos aparecerá lo siguiente.


Inspector
Tiles_spritesheet (Texture 2D) Import Settings
Open
Texture Type
Sprite (2D and UI)
Texture Shape
2D
Sprite Mode
Multiple
Pixels Per Unit
100
Mesh Type
Tight
Extrude Edges
1
Generate Physics Shape
Open Sprite Editor
Advanced
Wrap Mode
Clamp
Filter Mode
Point (no filter)
Aniso Level
1
Default
Max Size
2048
Resize Algorithm
Mitchell
Format
Automatic
Compression
Normal Quality
Use Crunch Compression
Revert
Apply
tiles_spritesheet
RGB
R
G
B
tiles_spritesheet
296x120 (NPOT) RGBA Compressed DXT5|BC3 sRGB 34.7 KB
AssetBundle
None
None

A lo que se ve acá, le seleccionamos un filtro de punto para que no se vea borroso, esto se ve en la parte de Advanced y filter mode.

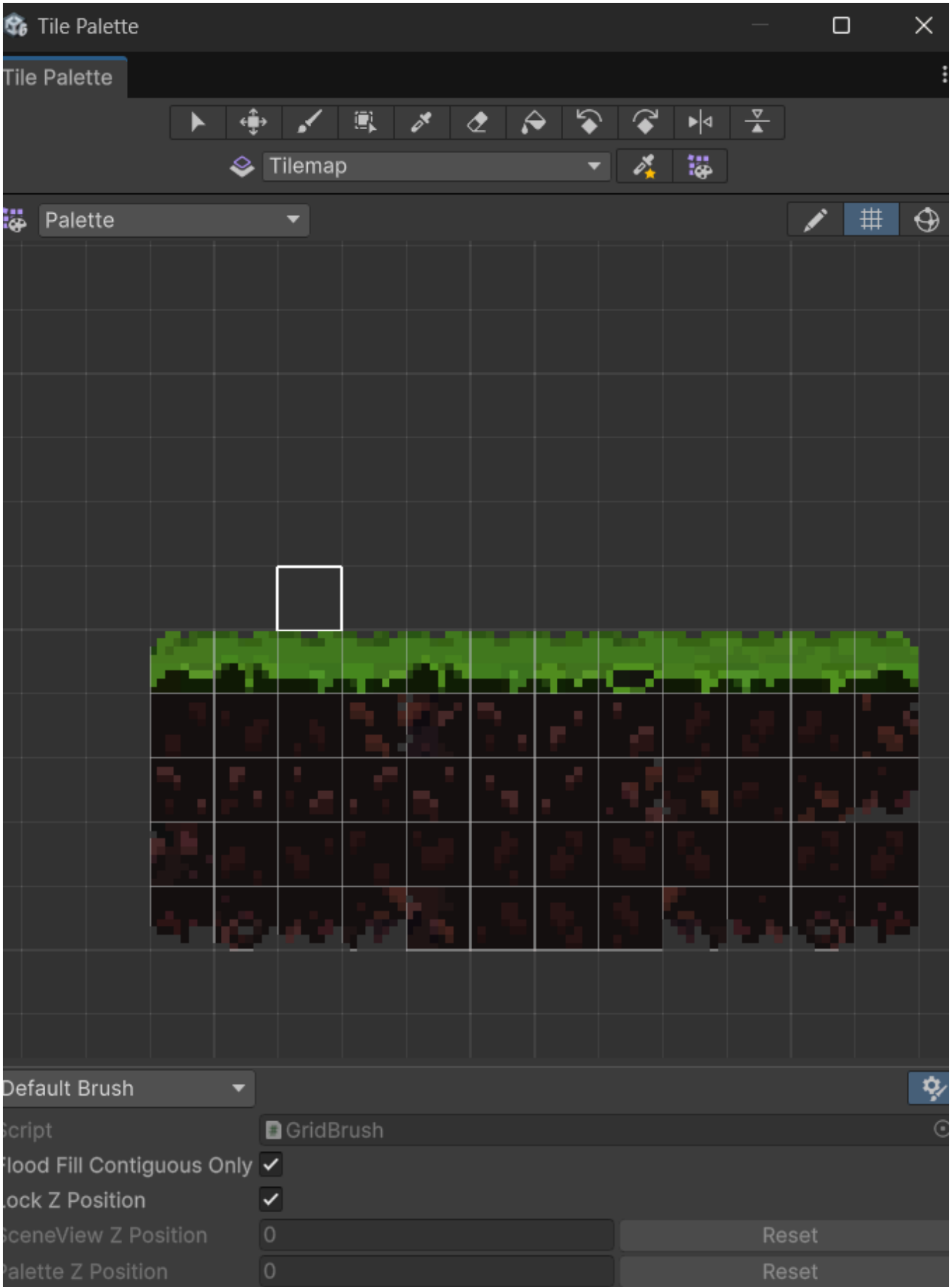
Con esto hecho, ahora vamos a la sección de window y le damos en 2D, luego en Tile palette y nos abrirá la siguiente ventana.



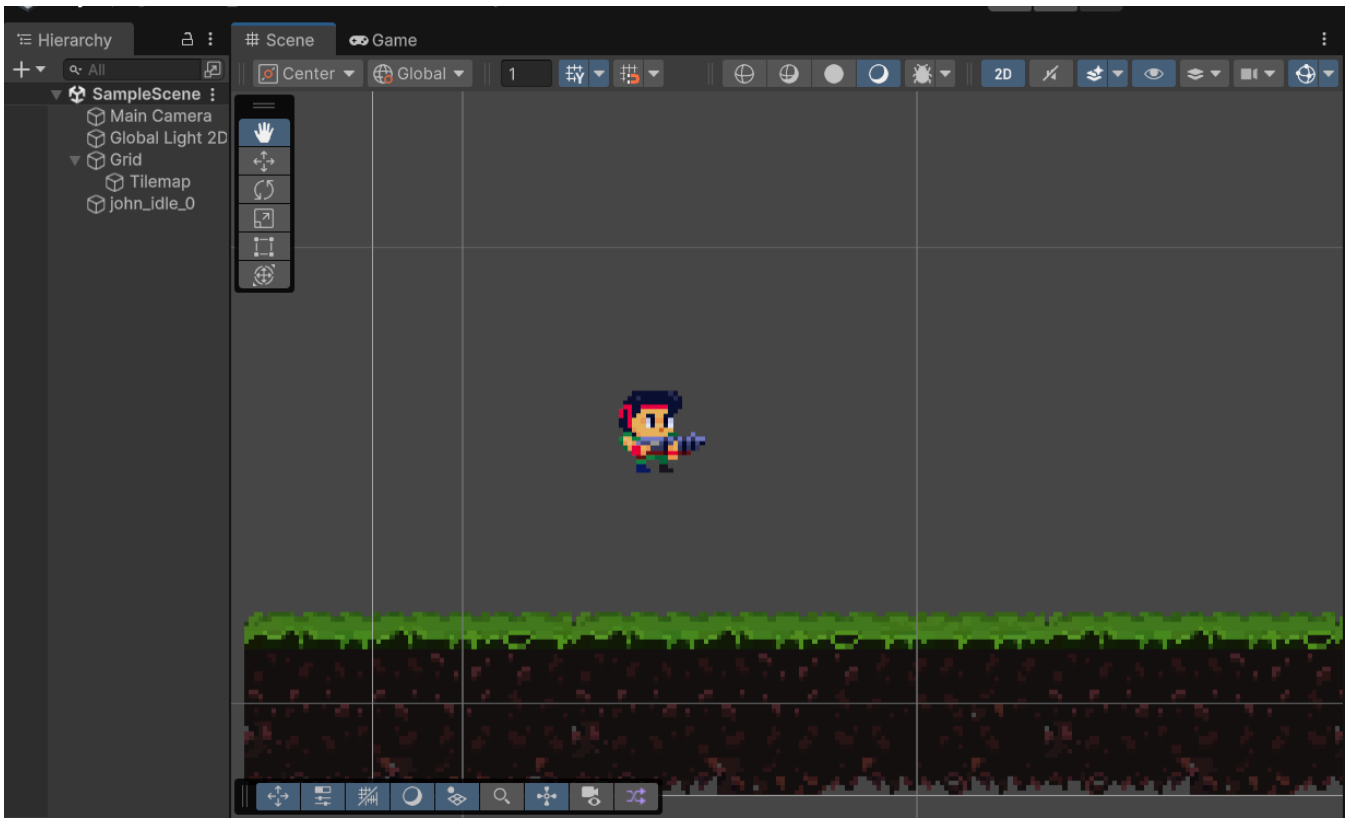


En esta ventana seleccionamos los cuadros que queremos para el tile.

Una vez seleccionado todo vamos al tile palette y lo ordenamos como lo necesitemos.

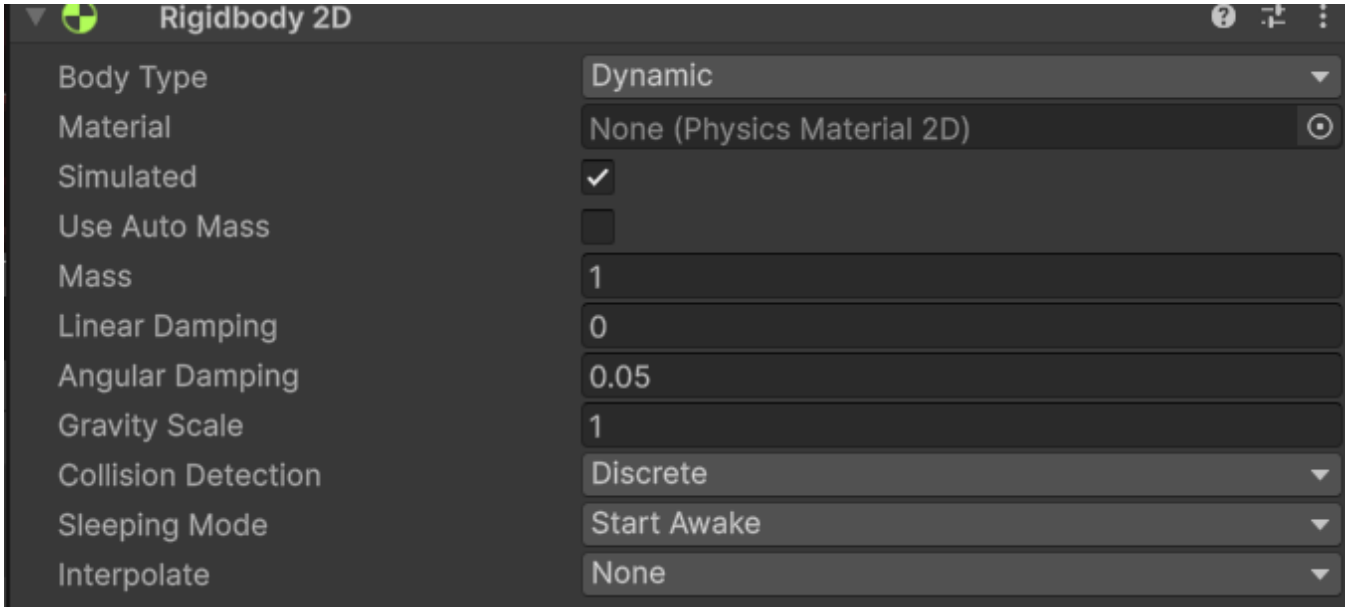

Tile Palette
Tile Palette
Tilemap
Palette
Default Brush
Script
GridBrush
Flood Fill Contiguous Only
Lock Z Position
SceneView Z Position
0
Reset
Palette Z Position
0
Reset

Una vez terminado, lo agregamos al campo visual.

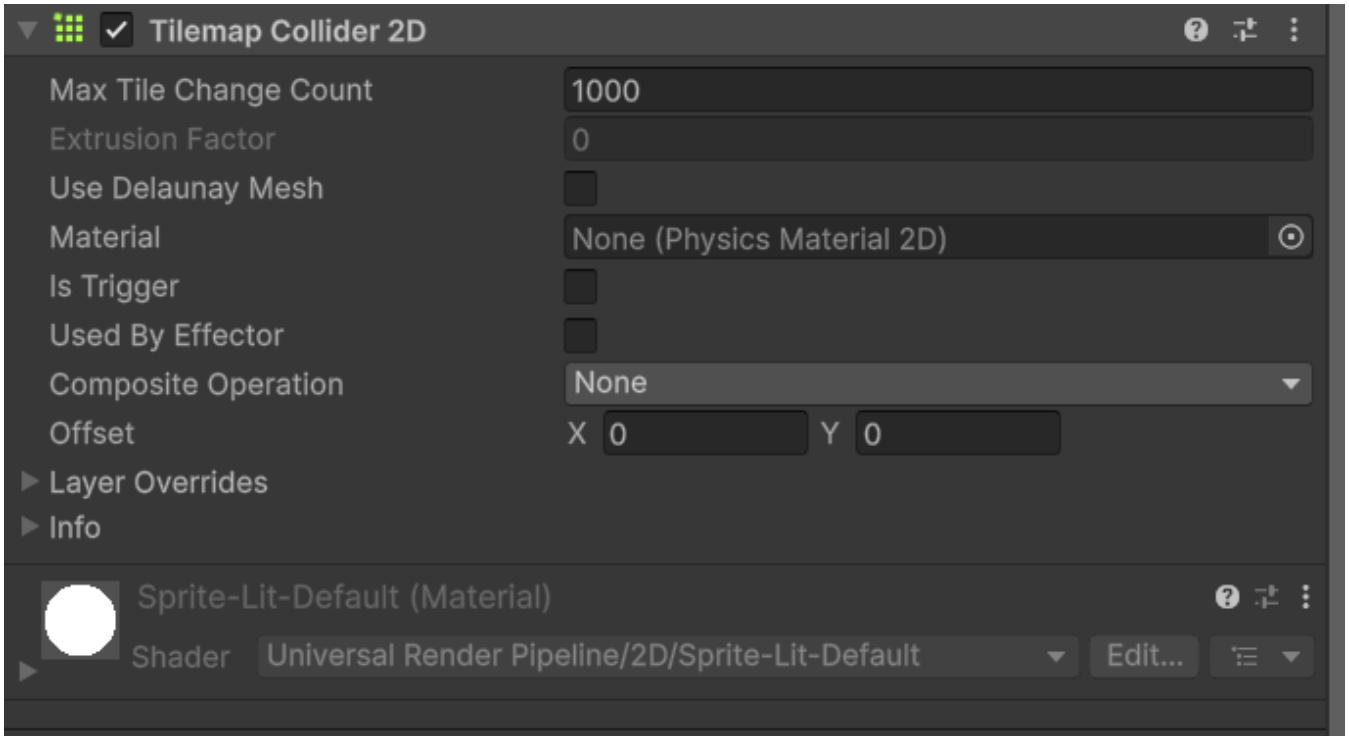


Para no hacer muy largo el documento, se hizo lo mismo con el personaje.

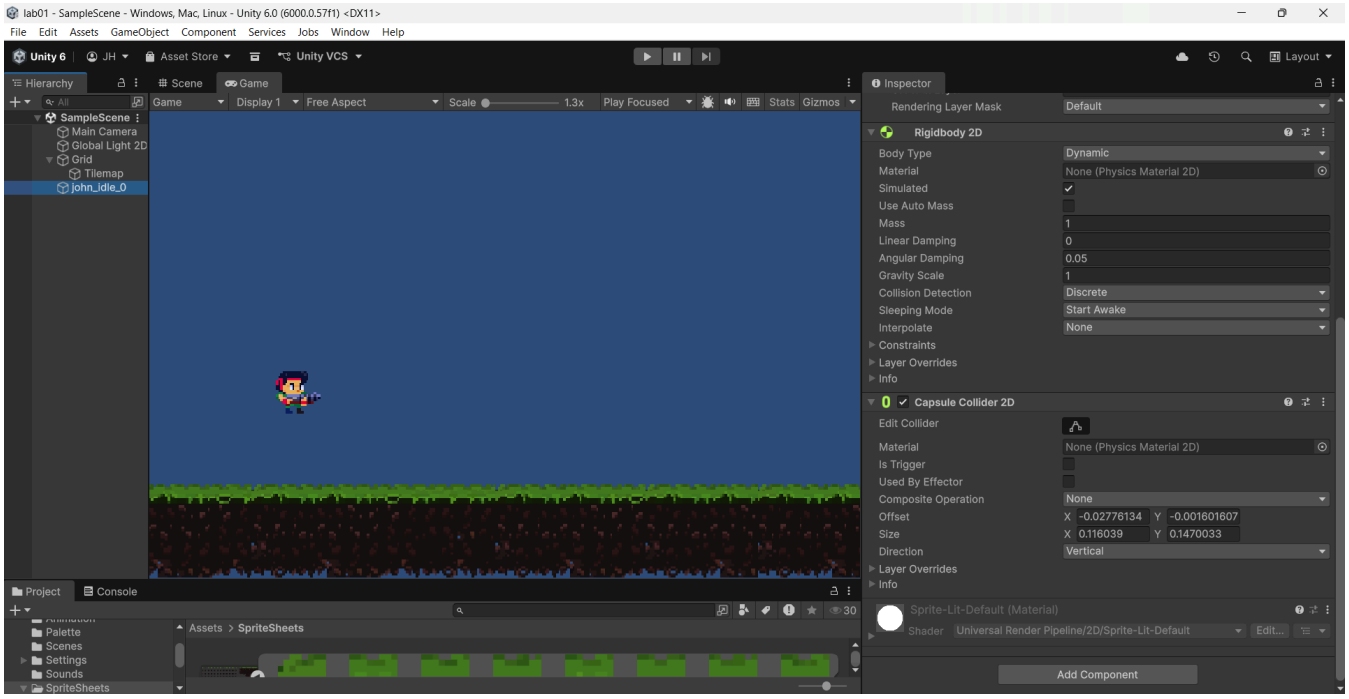
Al personaje le añadimos rigid body 2D para que tenga las físicas.



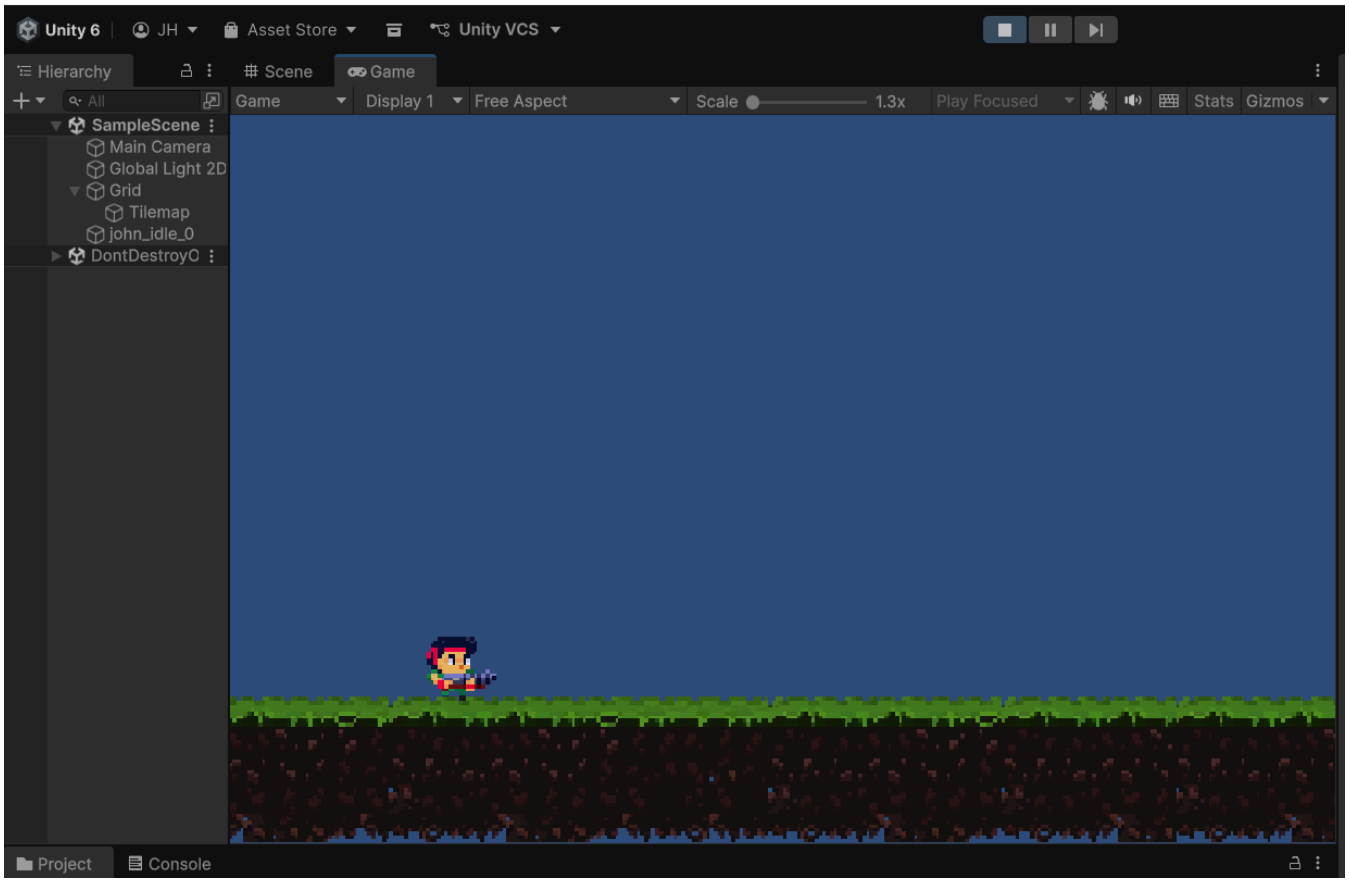
Al mapa le añadimos TIlemap Collider 2D para que también pueda ser detectado para la colisión.



También le añadimos collider al jugador, en este caso un collider de cápsula.



Una vez añadidos los colliders, el juego iniciado debería mostrar estos choques.



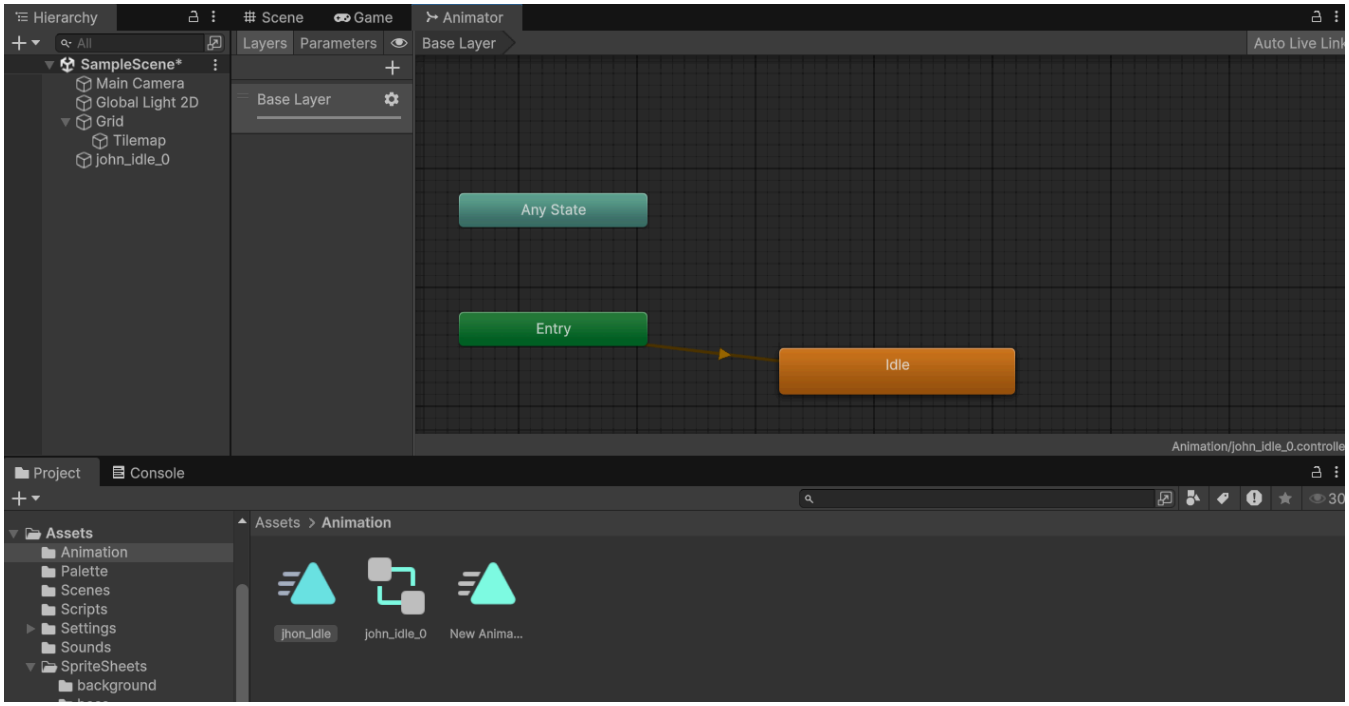
En ejecución, el personaje cae desde el cielo hasta el piso.

Ahora añadimos el script para que el personaje se pueda mover.

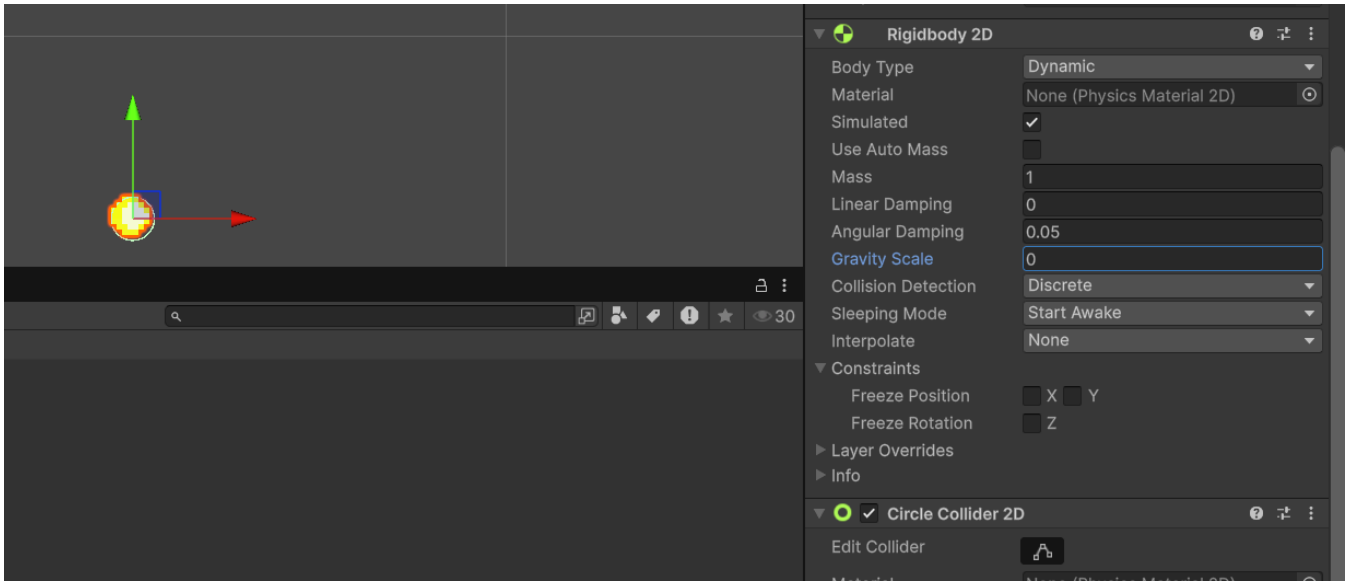
```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

Script de Unity | 0 referencias
public class jhon_movement : MonoBehaviour
{
    private Rigidbody2D Rigidbody2D;
    private float Horizontal;

    Mensaje de Unity | 0 referencias
    void Start()
    {
        Rigidbody2D = GetComponent<Rigidbody2D>();
    }

    Mensaje de Unity | 0 referencias
    void Update()
    {
        Horizontal = Input.GetAxisRaw("Horizontal");
    }

    Mensaje de Unity | 0 referencias
    private void FixedUpdate()
    {
        Rigidbody2D.linearVelocity = new Vector2(Horizontal, Rigidbody2D.linearVelocity.y);
    }
}
```

Con ese pedazo de código, considerando el que el personaje se tiene que mover a 60fps, esto arreglado con el FixedUpdate.

Ahora para la parte de animaciones del personaje, vemos el siguiente diagrama y le añadimos el movimiento de jhon_idle que es el conjunto de 5 sprites.



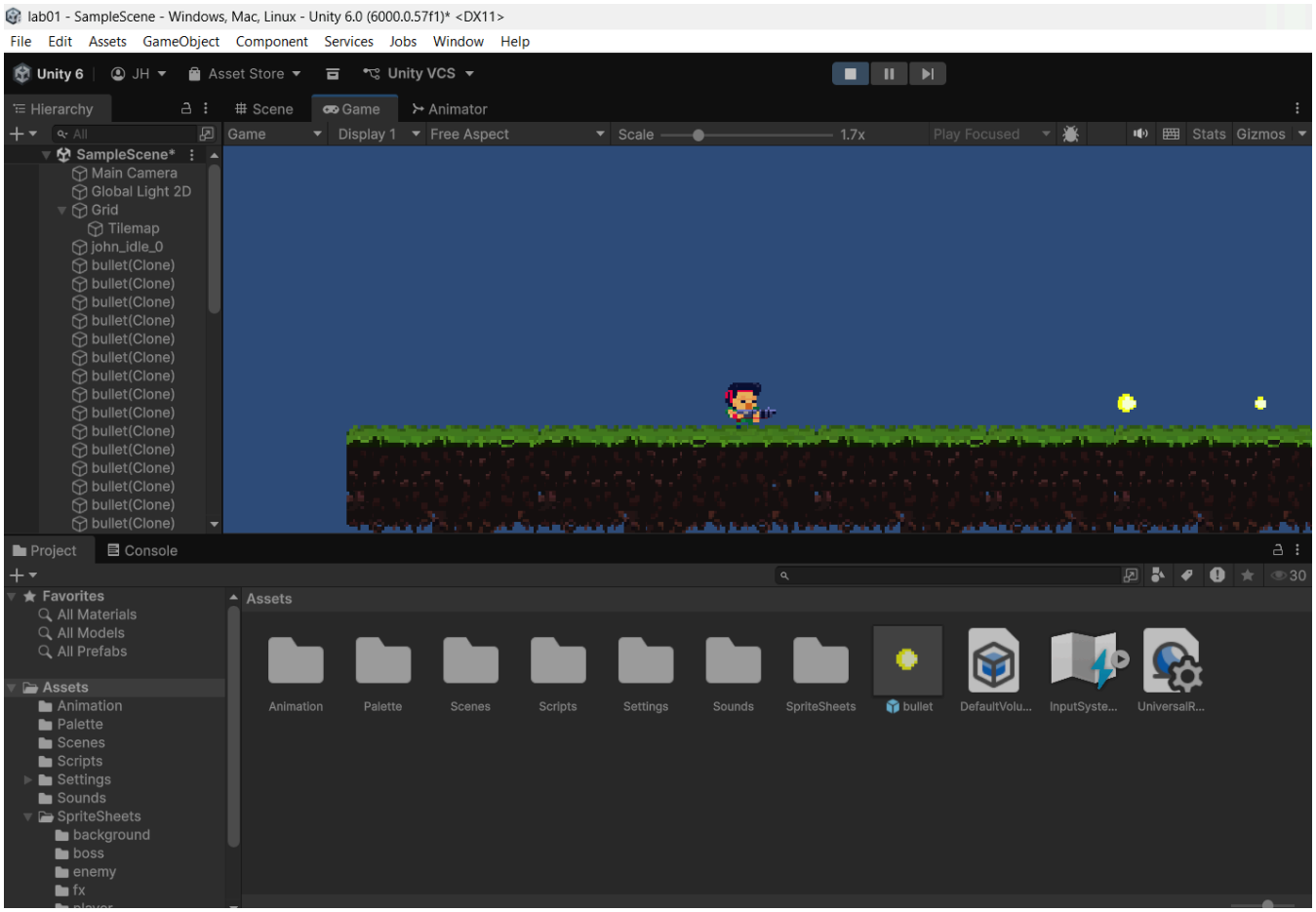
Una vez hecho eso, el personaje realiza animaciones.



igual con la bala, le añadimos el rigidbody y el collider para que colisione.

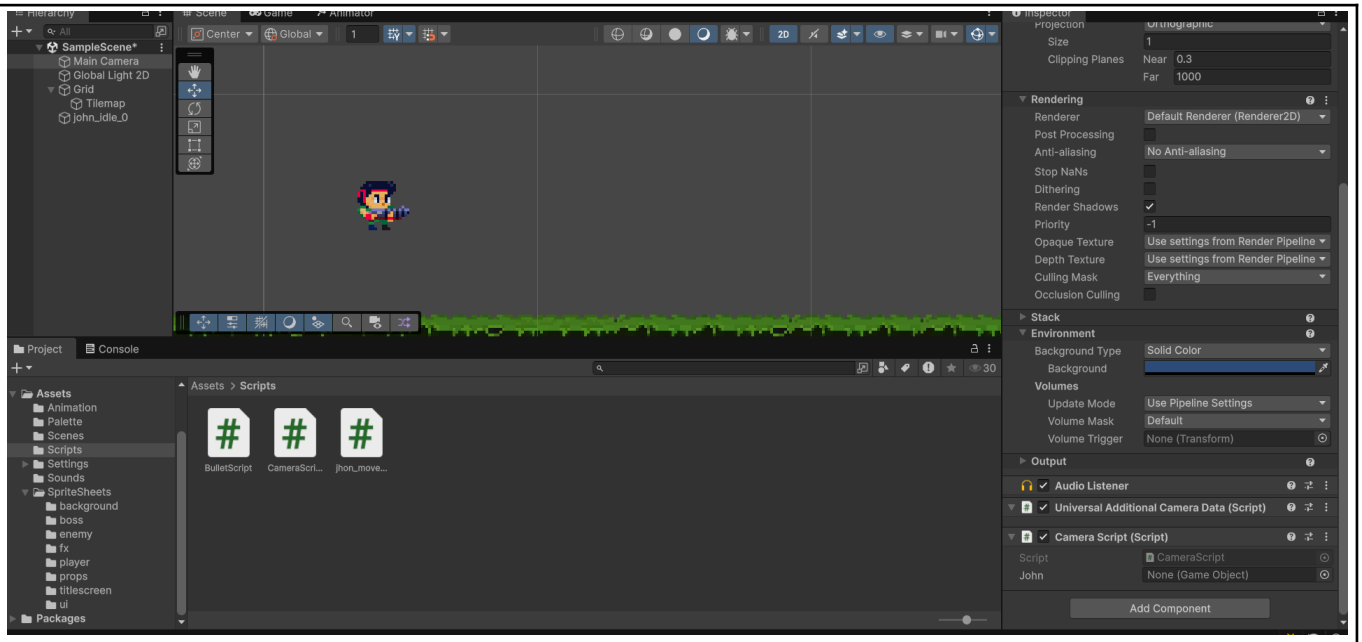
Luego con la función Shoot, hacemos que el prefab Bullet se inicie desde el personaje y le damos la dirección basada en la dirección que obtenemos también al presionar la tecla a o d.

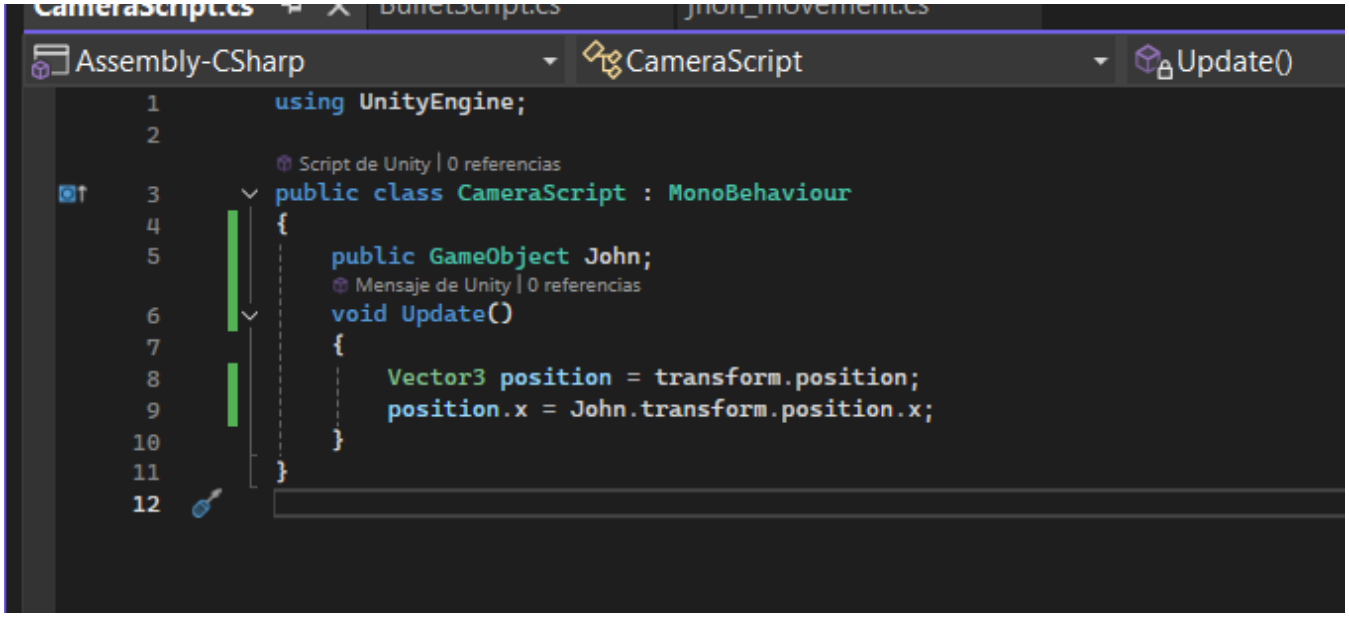
```
1 referencia
private void Shoot()
{
    Vector3 direction;
    if (transform.localScale.x == 1.0f) direction = Vector2.right;
    else
    {
        direction = Vector2.left;
    }
    GameObject bullet = Instantiate(BulletPrefab, transform.position + direction * 0.1f, Qu
    bullet.GetComponent<BulletScript>().SetDirection(direction);
}
Mensaje de Unity | 0 referencias
```



Una vez hecho eso, ya podemos ver que el personaje lanza las balas.

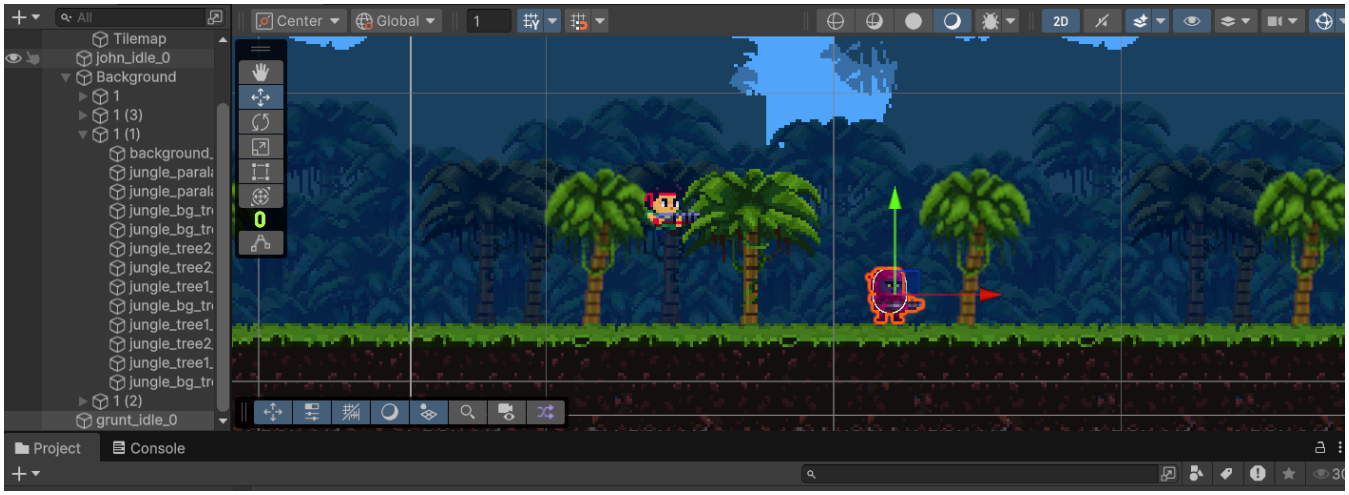
Ahora a la cámara le añadimos el script para que vaya con el personaje.





Añadimos el fondo y el primer enemigo. Un proceso muy similar al del jugador, solo que para el enemigo añadimos un script que calcula la posición en x del jugador y el enemigo, haciendo que siempre vea al jugador.

```
using UnityEngine;

Script de Unity | 0 referencias
public class GruntScript : MonoBehaviour
{
    public GameObject John;
    Mensaje de Unity | 0 referencias
    void Update()
    {
        Vector3 direction = John.transform.position - transform.position;
        if (direction.x >= 0.0f) transform.localScale = new Vector3(1.0f, 1.0f, 1.0f);
        else transform.localScale = new Vector3(-1.0f, 1.0f, 1.0f);
    }
}
```



Ahora que el enemigo está añadido y con la forma de disparar, nos falta añadirle una función importante a la bala que sería el on colission, de no ser usado, solo se detectaría como una colisión normal y no sucedería más que un movimiento en el jugador.

Con esta función vemos que si choca con el enemigo o con el jugador.

```
Mensaje de Unity | 0 referencias
private void OnCollisionEnter2D(Collision2D collision)
{
    jhon_movement john = collision.collider.GetComponent<jhon_movement>();
    GruntScript grunt = collision.collider.GetComponent<GruntScript>();
}
}
```

Eso detecta colisiones, pero faltaría añadir un sistema de vida.

Tanto en el jugador como en el enemigo añadimos el atributo de vida.

```
14    private float LastShoot;
15    private int Health = 5;
```

también la función hit para que reciban daño

```
1 referencia
74    public void Hit()
75    {
76        Health = Health - 1;
77        if (Health == 0)
78        {
79            Destroy(gameObject);
80        }
81    }
```

Finalmente en bala hacemos que sino se detecta el nulo, el personaje recibe daño.

```
Mensaje de Unity | 0 referencias
38    private void OnTriggerEnter2D(Collider2D collision)
39    {
40        jhon_movement john = collision.GetComponent<jhon_movement>();
41        GruntScript grunt = collision.GetComponent<GruntScript>();
42        if(john != null)
43        {
44            john.Hit();
45        }
46        if( grunt != null)
47        {
48            grunt.Hit();
49        }
50        DestroyBullet();
51    }
52 }
```

57 % No se encontraron problemas. Línea: 15 Carácte

Video:

https://drive.google.com/drive/folders/1BiANUpCxI3P2CAZqlwZBk5WHCawF8sIj

II. SOLUCIÓN DEL CUESTIONARIO

No hay un cuestionario

III. CONCLUSIONES

- Una actividad buena para poner en práctica lo que hicimos en semestres pasados.
- El que podamos implementar un poco de todo y no limitarnos solo al 3D nos ayuda a ver otras formas de desarrollo de juegos como el que apliqué en este caso.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[]https://www.youtube.com/watch?v=GbmRt0wydQU
[]https://didigameboy.itch.io/jambo-jungle-free-sprites-asset-pack/download/eyJleHBpcmVzIjoxNzc2ODI3NDAxLCJpZCI6MTUxNzE0fQ%3d%3d%2eZOk0YHJ7il1eY2vFT9pM9e1%2fh1M%3d